

模型论文里的基础设施护城河

为什么 DeepSeek-V4 的 § 3 才是最值得读的章节

—— 从 MegaMoE、TileLang 到 batch invariance 与异构 KV cache

摘要 · DeepSeek-V4 公布以来，社区几乎所有目光都集中在 § 2 Architecture: CSA、HCA、mHC、Muon。但读完整篇论文，真正构成不可复现护城河的不是 架构组合，而是 § 3 General Infrastructures 描述的整套训练—推理基础设施：MegaMoE 的 fine-grained EP 流水线、TileLang DSL、batch-invariant 与 deterministic kernel、FP4 QAT、Muon 的 ZeRO 适配、contextual parallelism、异构 KV cache 与 on-disk 存储策略。这些组件的复合效应，是 30K B200 集群 与 多年系统投入的产物，而不是任何单点算法可以解释的。本文按 "MoE 通信 / kernel 生产力 / 确定性训练 / KV cache 服务化" 四个维度，对 § 3 做一次系统的批判性阅读。

技术博客 · 批判性读后感 | 2026年4月 | 适合读者：模型基础设施、分布式训练、推理系统、GPU kernel、MoE serving 方向

模型论文里的基础设施护城河

为什么 DeepSeek-V4 的 § 3 才是最值得读的章节

SYSTEMS **MoE** **INFERENCE** DeepSeek-V4 技术报告批判性阅读 | 全文约 8500 字 | 阅读时间 25 分钟

DeepSeek-V4 论文 60 页，社区讨论几乎全集中在前 14 页：CSA / HCA 的混合 attention，mHC 的流形约束，Muon 的引入。但如果你做过大规模训练或推理 infra，你会知道这些都是论文里"看得见的部分"。真正决定 V4 能不能在 1.6T 参数 $\times$ 33T token $\times$ 30K B200 的尺度上跑通的，是 § 3 General Infrastructures 里那一组互相咬合的工程组件——而这套组件，开源社区在很长一段时间内都没法完整复现。

本文结构

- ▶ 0. 为什么 § 3 才是 V4 的真正护城河
- ▶ 1. MoE 通信：MegaMoE 与 fine-grained EP 流水线
- ▶ 2. Kernel 生产力：TileLang DSL 的"精确优先"哲学
- ▶ 3. 确定性训练：batch invariance、deterministic kernel 与 FP4 QAT
- ▶ 4. KV cache 服务化：异构布局、状态/经典分离、on-disk 三策略
- ▶ 5. 真正的护城河 —— 30K B200 与全栈协同

0 为什么 § 3 才是 V4 的真正护城河

读 DeepSeek-V4 paper，最容易做的事是看 § 2 Architecture 然后对几个新名词（CSA、HCA、mHC）做一组架构层面的"创新性"评论。但这其实是论文最容易 评估、也最容易被高估的部分。让我们先做一个不那么舒服的对比：把 § 2 里的每个组件溯源回它的来源。

V4 组件	实质来源	V4 的增量
MoE 主体	DeepSeekMoE (V3)	gating 改 Sqrt(Softplus); 前 3 层 Hash routing
CSA	NSA (Yuan 2025) + DSA (V3.2) 的组合	压缩 + 稀疏的串联 + grouped output projection
HCA	MLA-style 压缩 attention	更激进的压缩比 $m'=128$
mHC	Hyper-Connection (Zhu 2025)	加入 Birkhoff 多胞形约束 + Sinkhorn-Knopp
Muon	Jordan 2024 / Liu 2025 (已被 Moonshot 用过)	大规模工程化适配
MTP	V3 原样保留	无

这个表的意思不是说 § 2 没有价值——它有，组合式创新本身需要工程品味，把这么多前沿组件在 1.6T 尺度上联合优化跑通绝不是小事。但它意味着一个事：§ 2 的每一个组件，原则上都可以被一个有论文复现能力的团队拿走。权重也开源了，HuggingFace 上挂着 DeepSeek-V4-Pro 和 Flash 的完整 checkpoint。

那 V4 的"真正护城河"在哪里？我的判断是：在 § 3 描述的那套基础设施里。这些组件不像架构那么"亮眼"，但它们有几个共同特点：

- 几乎都是 first-of-kind 的解法，社区里没有现成对应物
- 互相耦合，单独抽出来都失去意义
- 需要 30K B200 规模的环境才能验证，而 30K B200 不是有钱就能买到的——硬件分配、机房、网络、电力都是壁垒
- 需要多年持续投入。MegaMoE 是 V3 时代 EP 优化的延伸，TileLang 是 PKU 长期合作的结果，3FS 至少经历了三代版本

论文摘要本身把这一点讲得很清楚——但是用工程师才会注意到的措辞：

"To enable efficient training and inference for DeepSeek-V4 series as well as productive development, we introduce several infrastructure optimizations. First, we design and implement a single fused kernel for MoE modules... Second, we employ TileLang... Third, we provide efficient batch-invariant and deterministic kernel libraries... Fourth, we incorporate FP4 quantization-aware training... Fifth, for the training framework... Finally, for the inference framework..."

"First / Second / Third / Fourth / Fifth / Finally" —— 在一段 abstract 里能出现六个并列的工程贡献，这是一个 systems-engineering paper 的语气，不是一个 algorithm paper 的语气。论文的真实自我定位被掩藏在了 abstract 后半段，但那才是 V4 的 backbone。

下面我按四个角度展开：MoE 通信、kernel 生产力、确定性训练、KV cache 服务化。每一节我会先讲问题是什么，然后看 V4 的解法，最后做批判性评价——哪些是真贡献、哪些是被 marketing 化的、哪些没说但应该说的。

1 MoE 通信：MegaMoE 与 fine-grained EP 流水线

1.1 EP 的根本难题

Mixture-of-Experts 的算力理论上无限可扩展（加 expert 就行），但工程现实里有一个无法绕过的瓶颈：expert parallelism (EP) 下的 all-to-all 通信。每个 token 要被 router 决策路由到 6（V4-Pro 是 6 个 activated expert, 384 个 routed expert）个 expert，这些 expert 分布在不同的 GPU 甚至不同节点上。一次 MoE 层的执行流程是：

```
Dispatch (all-to-all) → Linear-1 (GEMM) → SwiGLU → Linear-2 (GEMM) → Combine (all-to-all)
```

朴素实现里，这五步是串行的——通信时 GPU 算力闲着，计算时 NVLink 闲着。在 V3 那种 256 expert + H100 + 14T token 的尺度上，这个浪费还可以忍。在 V4-Pro 的 384 expert + 1.6T 参数 + 33T token + B200 尺度上，串行执行会让训练时间多一倍。

Comet (Zhang et al. 2025b) 的方案是粗粒度双路重叠：把 Dispatch 和 Linear-1 重叠、Linear-2 和 Combine 重叠。Figure 5 (b) 是这个方案，理论加速 1.42×。但 Comet 的局限是：当 Dispatch 比 Linear-1 长，或者 Combine 比 Linear-2 短时，重叠就失效一部分。

performance.

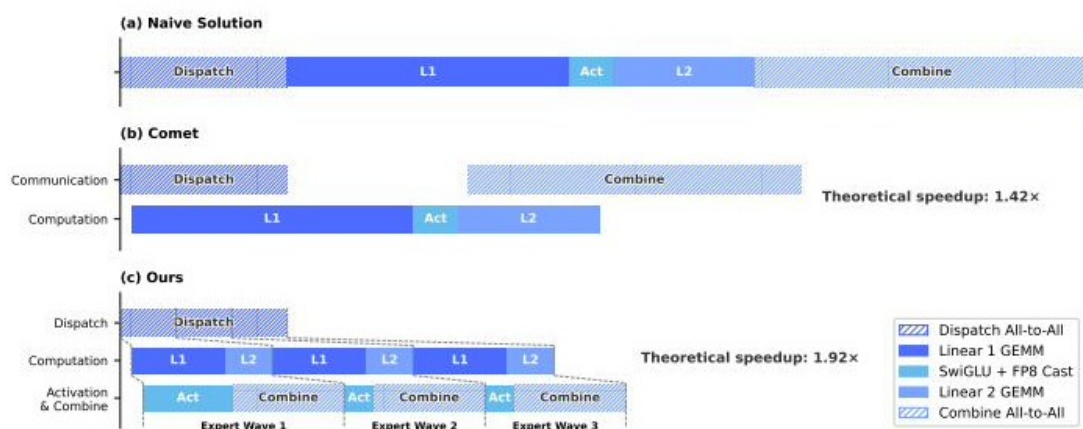


Figure 5 | Illustration of our EP scheme with related works. Comet (Zhang et al., 2025b) overlaps Dispatch with Linear-1, and Linear-2 with Combine, separately. Our EP scheme achieves a finer-grained overlapping by splitting and scheduling experts into waves. The theoretical speedup is evaluated in the configuration of the DeepSeek-V4-Flash architecture.

图 1 | DeepSeek-V4 paper Figure 5. 三种 EP 通信-计算重叠方案的对比。(a) Naive 串行；(b) Comet 的粗粒度双路重叠 ($1.42\times$ 加速)；(c) V4 的 wave-based fine-grained pipeline ($1.92\times$ 加速)。注意 (c) 中 wave 1 / wave 2 / wave 3 三组 expert 的计算和通信完全交错。

1.2 Wave-based pipeline: 把 expert 切成更小的浪

V4 的方案 (c) 思路是把 expert 切成更小的“波次” (waves)，每个 wave 只包含一小部分 expert。一旦某个 wave 的数据 dispatch 完成，立刻开始算那一小批，不必等其他 expert 的数据到位。论文里的稳态描述：

"In steady state, computation of current wave, token transfer for the next wave, and result sending of completed experts all proceed concurrently."

三路并行：正在算的 wave + 正在来的 wave + 正在走的 wave。理论加速 $1.92\times$ ，比 Comet 接近翻倍。这个思路 Aimuyo et al. 2025 和 Comet 自己 都触及过，但 V4 把它做到了 production scale 并开源——叫 MegaMoE，作为 DeepGEMM 的一部分发布在 PR #304。

关键 trade-off 是 wave 大小：

- wave 太小：每个 wave 的 GEMM 太小，浪费 tensor core
- wave 太大：退化到 Comet 的粗粒度重叠

论文没透露具体 wave 大小（这是 lab 经验值），只说在 NVIDIA 和华为 昇腾两套硬件上都做了 tuning。

1.3 一个被忽略的硬核分析：C/B = 6144 FLOPs/Byte

§ 3.1 后面有一段非常硬核的推导，社区基本没人讨论。论文给出 V4-Pro 的 计算-通信比（compute-communication ratio）：

每 token-expert pair：6hd FLOPs (SwiGLU 三次投影) | 3h bytes (FP8 dispatch + BF16 combine)

$$C / B \leq 2d = 6144 \text{ FLOPs/Byte}$$

这个数字的物理意义是：每字节 NVLink 数据，必须配套至少 6144 次 浮点运算，通信才能完全被计算掩盖。换算就是：每 GB/s 的 interconnect 带宽 能 hide 6.1 TFLOP/s 的计算。

把它对到 B200 上做校准：

硬件项	B200 数值	含义
FP8 tensor core	≈ 4.5 PFLOPs/s (dense)	V4 expert GEMM 的算力上限
NVLink 5 单向带宽	≈ 900 GB/s (双向 1.8 TB/s)	EP all-to-all 的通信带宽
实际 C/B	≈ 5000 FLOPs/Byte	低于论文要求的 6144

结论：B200 上 V4 的 EP 是略微通信受限的。这解释了 V4 的 两个激进选择：(a) FP8 dispatch + BF16 combine 的非对称精度——dispatch 数据量大，FP8 砍一半；combine 是聚合，要 BF16 保精度；(b) wave-based pipeline 是必需的，串行方案在 B200 上根本压不出 MFU。

1.4 给硬件厂商的四条建议

§ 3.1 末尾还有四条 "Observations and Proposals"——表面上是给硬件厂商 的建议，实质上是 DeepSeek 对未来 GPU 设计的 wishlist：

1. 带宽别再无脑堆。已经过 6144 FLOPs/Byte 平衡点的话，硅面积 应该花在算力而不是带宽上。
2. 电源预算（power headroom）必须够。极致 kernel fusion 让 compute + HBM + NVLink 同时打满，B200 的 1000W TDP 在 wave pipeline 下实际 触发 thermal throttling 是常见现象。
3. 低延迟跨 GPU signaling。V4 现在用 pull-based（接收方主动 RDMA 读），是 push-based 在 fine-grained wave 下 notification 太贵的 workaround。如果未来 NVLink 加低延迟 signaling 原语，push 模式更自然。
4. 替换 SwiGLU。SwiGLU 的 sigmoid 有 exp / division，且 gate projection 占 1/3 expert 参数。建议换成无 exp/div 的 element-wise 激活，省下的参数预算让 intermediate dim d 更大，进一步放松带宽要求。

注意第 4 条是建议而不是 V4 自己的做法——V4 仍然用 SwiGLU。而 § 4.2.3 的 "SwiGLU Clamping"（mitigating training instability）实际上已经 暴露了 SwiGLU 的 outlier 问题。把这两段串

起来读，就能看出 DeepSeek 内部对 SwiGLU 的判断已经在动摇——但 V4 这一代还没换。下一代很可能会换。

⚠ 批判性评价 — §3.1

原创性：中高。Wave-based pipeline 不是完全原创，但 V4 的实现细节（具体 scheduling、pull-based、与 SwiGLU 的 fusion）有自己的工程贡献。

开放性：好。MegaMoE 作为 DeepGEMM 的一部分已开源（PR #304），社区可以读源码学习。LMSYS / SGLang 已经在 day-0 集成。

留白：(1) 没有给 wave 大小的扫描结果；(2) 没有公布在不同硬件（H100 vs B200 vs Ascend）上的实测 MFU；(3) C/B = 6144 这个数字是 paper 仅给的，没有在不同模型尺度上的扩展（Flash 版的 d 不同，对应数字应不同）。

2 Kernel 生产力：TileLang DSL 的"精确优先"哲学

2.1 为什么 PyTorch ATen 撑不起 V4

论文 §3.2 开篇一句话：

"In practice, our elaborate model architecture would have resulted in hundreds of fine-grained Torch ATen operators."

V4 的架构（mHC + 混合 attention + lightning indexer + FP4 expert + RMSNorm on Q/K + 各种小 GEMM），如果用 PyTorch ATen 默认 op 实现，会有几百个细粒度 kernel。每个 kernel launch 在 H100/B200 上 5–10 微秒，kernel launch overhead 累积起来会成为 RL rollout 这种小 batch 场景的瓶颈。

主流应对方案有三种：

1. **手写 CUDA：**极致性能，但每次架构改动都要重写
2. **Triton：**易用，但在复杂 layout、warp 级控制上表达力受限，且默认 fast-math（追求吞吐而牺牲精度）
3. **CUTLASS：**性能好但局限于 GEMM 类，开发门槛高

V4 选择了第四条路——TileLang，一个北大团队主导、DeepSeek 深度参与的 DSL。它的卖点是"快速写 + 精确算"：像 Triton 那样可以几十行表达一个 fused kernel，但默认关掉 fast-math，要求 bit-identical 复现。

2.2 Host Codegen：把 CPU 开销压到亚微秒

这是 §3.2 最实用的一段：

"As accelerators continue to grow in performance, CPU-side orchestration overhead becomes increasingly prominent... CPU-side validation overhead drops from tens or hundreds of microseconds to less than one microsecond per invocation."

问题机制：GPU 越快，CPU 端的 kernel launch + 参数检查 + 调度 overhead 占比越高。Python 端的 `assert x.shape == ...` 每次几十微秒。对一个 10 微秒就能跑完的小 kernel，CPU overhead 反而成了瓶颈。

TileLang 的 Host Codegen 方案：

- 编译时从 DSL 前端提取 type / shape / stride 元信息
- 和 device kernel 一起生成一个 C++ host launcher
- 运行时通过 TVM-FFI 做 zero-copy tensor interop
- 所有参数检查由生成的 C++ 代码完成，不走 Python 解释器

这是个常被低估的 systems gain。在 H100 上一个 RMSNorm 可能 3 微秒跑完，CPU 端 50 微秒在 launch；切到 B200 同样 3 微秒（B200 算得更快但 RMSNorm 不会更快多少，因为它是 memory-bound），CPU 端的 50 微秒就成了 16× 的 overhead 比例。Host Codegen 把这个比例压回 0.3×。

2.3 Z3 SMT solver：编译器里的形式化整数推理

另一个非常硬核的设计：TileLang 集成 Z3 SMT solver 做整数表达式的形式化推理。这听起来抽象，但在 kernel 编译里有实际意义：

- **vectorization**：要证明某个 loop 的 stride 是常数，才能 vectorize
- **memory hazard detection**：要证明两次写入不会冲突
- **bound analysis**：要证明 array index 在合法范围

过去这些用启发式或保守估计，错过很多优化机会。Z3 给的是非线性整数算术的判定能力（QE_NIA），可以处理 vectorization over variable tensor shapes 这类复杂情况。论文说编译时间 overhead 控制在 "just a few seconds"，对 production kernel 完全可接受。

2.4 默认精确，opt-in 优化

这一段是 TileLang 真正区别于 Triton 的设计哲学：

"We therefore prioritize accuracy by default: fast-math optimizations are disabled at the compiler level, and precision-affecting approximations are provided only as explicit, opt-in frontend operators (e.g., `T.__exp`, `T.__log`, and `T.__sin`). Conversely, when strict IEEE-754 semantics are required, TileLang provides IEEE-compliant intrinsics with explicit rounding modes (e.g., `T.ieee_fsqrt`, `T.ieee_fdiv`, and `T.ieee_add`)."

对照 Triton 的默认行为：

方面	Triton 默认	TileLang 默认
fast-math	开（追求性能）	关（追求精度）
exp / log / sin	硬件近似	IEEE-754 严格，opt-in 才能用近似
rounding mode	编译器决定	显式指定
bitwise reproducibility	不保证	对齐 NVCC 算法简化规则，可对手写 CUDA bit-identical

这种"严格优先"的态度看起来是绕远路，但在 $1.6T \times 33T$ token 的训练 里它有现实意义：任何数值漂移都会被放大成 **loss spike**，而 **loss spike** 在 30K B200 上每出一次都是几十万美元的成本。Spike 出现时如果 `kernel` 不可复现，根本没法定位。这一节读起来像金融科技或航天软件的精度 控制规范，但对 frontier-scale LLM training 来说，确实需要这个级别的严格性。

⚠ 批判性评价 — § 3.2

定位：TileLang 不是"另一个 Triton"，而是 Triton 的 production-grade 升级版——更 opinionated，更严格，学习曲线更陡，但用对了场景下更可靠。

开源价值：高。TileLang 是独立开源项目（[tile-ai/tilelang](#)），你可以用它写自己的 kernel。论文 ICLR 2026 投稿（Wang et al. 2026）。LMSYS / SGLang 已经把 TileLang 用在 mHC kernel 集成里。

留白：没有给 TileLang vs Triton/CUTLASS 在 V4 实际 kernel 上的端到端 benchmark。SOTA 学术 paper 一般会给 GEMM、attention 几个 case 的对比，V4 paper 完全省略了。

3 确定性训练：batch invariance、deterministic kernel 与 FP4 QAT

3.1 一个看似冷门、实则致命的问题：batch invariance

这一节是 § 3 全篇我最佩服的工程严谨性。先讲清楚问题：

"Batch invariance ensures that the output of any given token remains bitwise identical, regardless of its position within a batch."

用人话说：同一个 prompt，单独喂进模型 vs 和 100 个其他 prompt 一起 batch 喂进去，结果应该完全一样（bit-identical）。这听起来是常识，但 主流 LLM 推理栈（vLLM, SGLang 旧版本）不满足这个性质。

原因有两个常见 GPU 优化模式破坏了 batch invariance:

- **Split-KV attention (FlashDecoding)**: 长序列 attention 拆给多个 SM 并行算 partial sum, 最后 merge。merge 顺序依赖 SM 调度, 不同 batch size 下调度不同, 浮点不结合律导致结果有 bit 差异。
- **Split-K GEMM**: 矩阵乘法把 K 维切成多段并行算再 sum, 同样的 associativity 问题。

这两个优化在单次推理时性能提升 20-50%, 但让模型输出依赖 batch shape。

这个问题在 RL post-training 场景下是**致命的**: rollout 阶段(生成数据) batch 大, training 阶段(policy update) batch 不同, 训练时模型"看到的"分布和 rollout 时模型"生成的"分布不一致。GRPO/PPO 这类 on-policy 算法的数学假设被轻微违反, 训练可能发散。Thinking Machines Lab 在 2025 年 9 月的博客 "[Defeating Nondeterminism in LLM Inference](#)" 里第一次系统讲清楚了这个问题, LMSYS / SGLang 在 2025 年底跟进集成了 batch-invariant kernel。

V4 把这个问题作为系统级目标内置到了 infra 里, 而不是后期 patch。这是巨大的差别。

3.2 Attention 的 dual-kernel 解

V4 的 attention batch invariance 解法很巧:

- **Kernel 1**: 单 SM 算一个完整序列。GPU 利用率好, 但 wave-quantization 的最后一波 (partially-filled) 会拖慢 latency。
- **Kernel 2**: 多个 SM 算一个序列 (解决 wave-quantization 的尾部)。但**关键**: 通过 distributed shared memory 跨 SM 高速交换, 并且**累加路径和 Kernel 1 完全对齐**, bit-identical 输出。

这是个让我读完拍腿的设计: 他们没有放弃 Kernel 2 的性能, 也没有放弃 Kernel 1 的严格性, 而是**设计了第二个 kernel 让它在数值上等价于第一个**。这需要在 thread block clusters 这一硬件特性上深度作业, 是 B200 (Hopper + Blackwell 才有 thread block cluster) 才方便做的设计。

3.3 Determinism: 三个非确定性来源 → 三个对应解

论文清晰地点出训练 backward pass 的三个非确定性来源:

来源	问题机制	V4 的解
Attention backward	稀疏 attention 的 KV gradient 用 atomicAdd 累加, 浮点非结合律破坏复现	每个 SM 用独立 accumulation buffer; 最后做 确定顺序 的全局求和
MoE backward	多个 SM 跨 rank 写到同一接收端 buffer, 写入位置协商引入非确定性	每个 rank 内部 token order 预处理 + 跨 rank 的 buffer 隔离
mHC 中的 GEMM	mHC 的 output dim 只有 24, 小 batch 必须 split-K, naive split-K 非确定	把每个 split 部分单独输出, 后续 kernel 做确定 reduction

这三个解都是"问题一解法"成对出现的，而且每个都是该领域里有 专门发现才能写出来的细节。Mhc 那个尤其精妙——你必须真的训过 mHC 才会 发现 24 这个 output dim 让 split-K 不可避免地引入非确定性，否则根本想不到 这是问题。

3.4 FP4 QAT：精度生态的最后一环

§ 3.4 看起来是独立一节，但放在确定性这条线下读才完整。V4 把 FP4 (MXFP4) 量化用在两个地方：

- **MoE expert weights**: 参数量大头，量化收益最大
- **CSA 的 indexer QK path**: activation 量大但精度敏感度低（只用于 top-k 选择），FP4 加速 attention score 计算

关键设计点是"lossless FP4-to-FP8 dequantization": 训练时 forward 用 FP4 模拟（先 dequant 到 FP8 再算 GEMM），backward 直接对 FP8 weight 求梯度并传播到 FP32 master weight（等价于 STE 透过 quantize 节点）。推理和 RL rollout 直接用 native FP4，不模拟——这保证 training 和 inference 的数值完全一致，不会出现"训练时 BF16，部署时 FP4"的分布漂移。

把整个数值精度生态串起来：

组件	训练精度	推理精度	设计意图
MoE expert weights	FP4 模拟（FP4→FP8 dequant）	原生 FP4	训-推一致
CSA indexer QK	FP4 + index score BF16	FP4 + BF16	top-k 加速
Main attention QKV	FP8 / BF16	同	精度敏感，不量化
mHC	BF16（output dim 24，FP4 无意义）	同	小 GEMM 不需 FP4
RMSNorm Embedding	/ BF16	BF16	数值稳定 / 语义敏感
梯度同步（MoE）	BF16 stochastic rounding, FP32 累加	—	通信减半，避免累加漂移

这是一个非常细致的"分层精度"设计，比 OpenAI gpt-oss-120B 那种" expert 整体 FP4"的简单做法成熟得多。每个组件的精度是按它的 数值敏感度 + 参数量 + 计算特征三个维度综合定的，不是一刀切。

⚠ 批判性评价 — § 3.3 + § 3.4

原创性：高。Batch invariance + deterministic kernel + FP4 QAT 三者 联合，做到 bit-identical training/inference，是目前开源 lab 里没有别人做到的级别。Thinking Machines 的工作更早但停在 inference；V4 把整套延伸到了训练。

战略意义：在 OPD (On-Policy Distillation, § 5.1) 里，teacher 多达 10+ 个、vocab > 100K、训练 step 长达数万次。没有 batch invariance + determinism，OPD 训练几乎不可能稳定收敛。所以这一节不是孤立的工程加分项，而是 V4 post-training paradigm 能跑起来的**必要条件**。

留白：(1) 没有量化数据：deterministic 模式相对非 deterministic 慢多少？SGLang 的数据是 ~34%，V4 的呢？(2) FP4 QAT 的 "lossless dequant" 依赖 FP4 子块 scale factor 的最大/最小比值不超过某阈值——论文说 "empirically verified"，但没给阈值、违反率、违反时的误差量级。这是可验证但未验证的声明。

4 KV cache 服务化：异构布局、状态/经典分离、on-disk 三策略

4.1 V4 的 KV cache 异构性

V4 的 attention 不是单一类型，每一层 stack 里至少有**五种**不同的 KV：

1. CSA main KV：每 m=4 token 压一个 entry，FP8 存储
2. CSA indexer KV：每 m=4 token 压一个 entry，indexer head dim = 128，FP4 存储
3. HCA KV：每 m'=128 token 压一个 entry
4. SWA KV：每 token 一个 entry，128 token 滑动窗口
5. Uncompressed tail KV：未达压缩阈值的 token 在 buffer 等待

这些 entry 的更新规则、淘汰策略、大小都不同。vLLM 的 PagedAttention 基本前提（所有 layer 大小一致、固定 page）在 V4 里失效。社区里目前最通用的混合 KV 方案 Jenga (SOSP 2025) 能处理"不同 layer 不同 attention"，但处理不了"单层内多种 KV 共存"——而 V4 的 CSA 层内部就有 4 种 KV。

这意味着 V4 必须自己设计 KV cache 管理，没有现成方案。

4.2 双层结构：State Cache vs Classical KV Cache

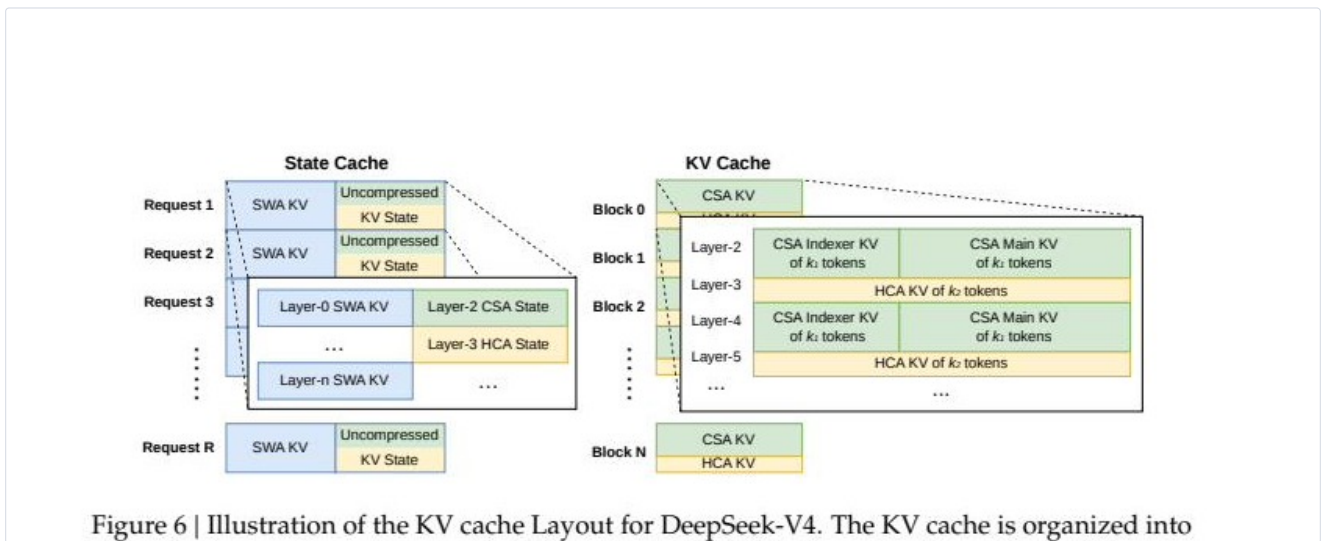


图 2 | DeepSeek-V4 paper Figure 6。KV cache 被组织成两个独立的池子。左：State Cache（每个 request 一个固定大小 block，存 SWA KV + uncompressed tail）。右：Classical KV Cache（每个 request 多个 block，每 block 覆盖 $\text{lcm}(m, m')$ 个原始 token，产生 $k_1 = \text{lcm}(m, m')/m$ 个 CSA 压缩 entry 和 $k_2 = \text{lcm}(m, m')/m'$ 个 HCA 压缩 entry）。

V4 的设计抽象：把 KV cache 拆成两个池子，每个内部规则简单。

State Cache（图左）：

- 存 SWA KV 和 uncompressed tail tokens
- 每个 request 一个固定大小 block
- 类比为 state-space model：数值只取决于当前位置，不是历史累积
- 可以 preemptively evict（丢了能从 checkpoint 重算）

Classical KV Cache（图右）：

- 存所有 append-only 的压缩 KV（CSA + HCA）
- 每个 request 多个 block，动态扩展
- 每 block 覆盖 $\text{lcm}(m, m')$ 个原始 token —— V4-Pro 是 $\text{lcm}(4, 128) = 128$
- Block 内部产生 $k_1 = 32$ 个 CSA entry + $k_2 = 1$ 个 HCA entry

这个 lcm 选择不只是数学美学，而是架构-infra 协同设计：让高性能稀疏 attention kernel 可以假设 block size 固定，loop bounds 编译时就确定。论文不点破但你能读出来—— m 和 m' 的选择部分是为了 lcm 友好。如果 $m=3$ ，lcm 变成 384，block 管理会更尴尬。所以这是个“算法选择被 infra 反向约束”的微妙例子。

4.3 On-disk KV cache 的 SWA 三策略

V4 的 1M context 让 shared-prefix caching 不是可选项而是必选项。一个 agent 任务的典型 context 分布是：

```
System prompt + tool definition = 10K token
历史对话                      = 50K token
当前 observation              = 5K token
-----
共享前缀 = 60K token, 独立部分 = 5K token
```

每次请求重新 prefill 60K token 是不可接受的。On-disk KV cache 就是把已经 prefill 好的 KV 存到 SSD，下次相同 prefix 直接 load。

CSA / HCA 的 KV 是 append-only，存到 disk 简单：直接写，下次按完整 压缩 block 边界读，tail 部分（不足 m 或 m'）重算几个 token。

但 SWA KV 是麻烦：未压缩、每层都有，体量大约是压缩 KV 的 8 倍。论文给了三个策略，每个对应不同的 storage / compute trade-off：

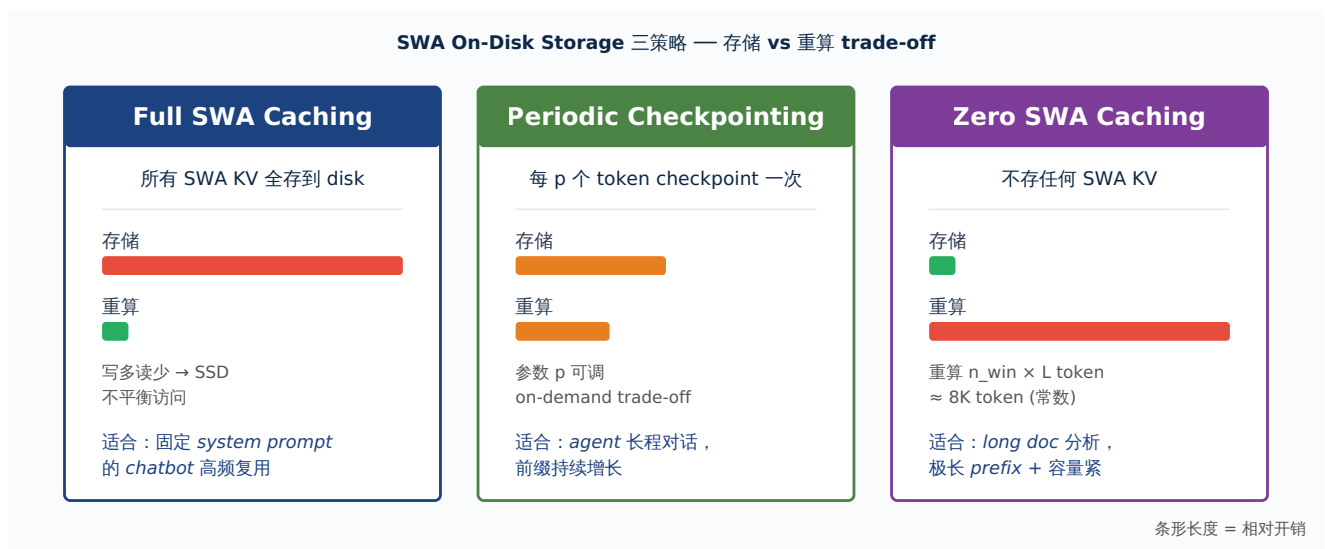


图 3 | V4 paper §3.6.2 的 SWA 三种 on-disk 存储策略对比。每种策略本质是在存储开销和重算开销之间选不同的 trade-off 点。V4 不强推某一种，而是把选择权留给部署方——这是对的工程态度。

Zero SWA 的重算链尤其有意思。SWA KV 的恢复链是逐层级联的：layer L 的最后 n_{win} 个 SWA 需要 layer L-1 的最后 n_{win} 个 hidden state，后者又需要 layer L-1 forward，依赖 layer L-1 的 SWA + CSA/HCA。每层向前 n_{win} 个 token 才能重建。L 层下来，总重算量 $\approx n_{win} \times L$ token。

V4-Pro: $128 \times 61 = 7808$ token 重算。不管 prefix 多长，重算量都是常数——对 100K prefix 是 8% 重算，对 1M prefix 是 0.8% 重算。这是 Zero SWA 的关键吸引点：重算复杂度对 prefix 长度不敏感。

4.4 对 agent 系统设计的启发

这一节是全篇我觉得对外部团队最有 takeaway 价值的：

1. **append-only 状态 vs rolling 状态的抽象分离**。Agent 系统里 tool call history 是 append-only, "当前心智状态"是 rolling——这两类用不同 存储/复用策略, 比 one-size-fits-all 干净得多。
2. **给用户三种策略 + 暴露 knob, 而不是追求最优自动决策**。流量 特征千差万别, 工程师选不出"全局最优"。Zero / Periodic / Full 是 Pareto 前沿, 让用户根据自己的流量特征选。
3. **lcm 技巧对 block-aligned 数据结构有普适价值**。当一个系统 有多种 granularity 共存 (如 4-token block 和 128-token block), lcm 是个 干净的对齐机制。

⚠ 批判性评价 — § 3.6

原创性：State cache vs classical cache 的分离是 V4 原创。Three SWA strategies 都不是新概念 (cache hierarchy 设计的 textbook trade-off), 但把它们三个都实现并 ship, 并清楚标出适用场景, 是工程师友好的做法。

没做的工作：(1) Prefix 的 fuzzy matching (只支持 exact prefix hit) ; (2) cross-sequence deduplication; (3) 自动策略切换。这些都是未来方向, V4 没做 合理——现在复杂度已经够。

5 真正的护城河 —— 30K B200 与全栈协同

把四节的内容拼起来, 我们得到一张"V4 真正的护城河图":

维度	组件	开源现状	复现门槛
MoE 通信	MegaMoE / fine-grained EP / DeepEP v2	已开源 (DeepGEMM PR #304)	需要 NVLink + 30K GPU 规模才能 tune wave 大小
Kernel 工具链	TileLang DSL + Z3 SMT 集成	独立开源 (tile-ai/tilelang)	学习曲线陡, 迁移现有 codebase 成本高
数值精度	batch-invariant + deterministic kernel + FP4 QAT	未开源 (idea 公开)	需要从零写一整套 kernel, 每个 op 都要保证 bit-identity
训练框架	Muon × ZeRO 混合策略 + tensor-level checkpointing + 2-stage CP	未开源	是 V4 训练框架的内部组件, 无独立 ship
推理框架	异构 KV cache + on-disk 三策略	SGLang 已 day-0 集成 (但靠 SGLang 自己实现)	设计公开, 工程实现可借鉴
分布式存储	3FS (teacher offload, on-disk KV)	已开源	部署 Lustre/Ceph 替代成本高
Sandbox infra	DSec (agentic AI 沙箱)	未开源	三组件 Rust 实现 + custom RPC, 重写工作量大

注意一个 pattern: 越底层的组件越倾向于开源 (kernel、文件系统), 越靠上的组件越倾向于闭源 (训练框架、agentic sandbox)。这不是偶然, 反映 DeepSeek 对自身 moat 的定位:

- 底层 kernel 开源——给社区 goodwill, 间接让 V4 模型在 vLLM / SGLang 生态里部署得更顺, 反过来给 DeepSeek 引流。
- 训练框架闭源——这是 V4 能训出来的真正秘方, 开源等于把 know-how 直接交出去。

这套全栈基础设施本身就是 V4 的最大贡献, 而不是 § 2 的架构创新。原因再列一遍:

1. 架构是组合式创新, 每个组件都有公开来源
2. 权重已开源——任何有 100 张 B200 的团队都可以 fine-tune
3. 但要从零训一个 V4-Pro, 必须有这套 infra——少一个组件训练就崩
4. 这套 infra 的整合, 需要 30K B200 + 多年 systems team 投入——这是资本和时间双重门槛

给 infra 工程师的 takeaway

✓ 如果你做 MoE serving

读 MegaMoE 源码 (DeepGEMM PR #304)。Wave-based pipeline 的具体调度策略是 production-grade 的参考实现。注意 wave 大小是要在自己硬件上调的, 不能照搬 DeepSeek 的值。

✓ 如果你做 GPU kernel 开发

试用 TileLang, 特别是它的 host codegen 和 IEEE-754 严格模式。Triton 在 production 场景的精度问题被低估, TileLang 的 "default-precise, opt-in fast" 是更成熟的工程态度。Z3 集成对 vectorization 优化有直接收益。

✓ 如果你做 RL/RLHF infrastructure

batch invariance 不是可选项。如果你的 rollout 和 training 用了不同 TP 配置, 结果一定不 bit-identical——这会 silently 把 on-policy RL 变成 off-policy RL。LMSYS 已经把 batch-invariant 集成进 SGLang, 可以直接用; 要做更激进的, 读 V4 的 dual-kernel attention 设计。

✓ 如果你做长上下文推理 / agent serving

把 "append-only 状态" 和 "rolling 状态" 明确区分, 分别用不同 cache 池子管理。Shared prefix 缓存的三策略 (Full / Periodic / Zero) 是 Pareto 前沿, 根据你的流量特征选。1M context 的可用性主要靠 on-disk KV, 不靠 GPU memory。

✓ 如果你做训练框架

Muon + ZeRO 不能直接套 DeepSpeed, 需要自己写 bucket assignment (V4 用 knapsack)。Tensor-level checkpointing 比 module-level 节省 activation memory 显著, PyTorch 原生 `torch.utils.checkpoint` 粒度太粗。Contextual parallelism 的两阶段设计 (neighbor 通信 + 全局整合) 是处理边界依赖架构的通用 pattern。

结语

V4 paper 60 页, §2 占 8 页, §3 占 8 页。从篇幅看是平的; 从信息密度看, §3 远高于 §2。从可复现性看, §2 的内容社区可以在 6-12 个月内追上 (已经有团队在做 mHC 和 hybrid attention 的复现实验), 而 §3 的整套生态系统社区追不上——它需要的不是论文写得清楚, 而是 30K B200 集群、多年 systems 团队、和一个愿意开源底层 kernel 但保留训练框架的战略选择。

读 V4 paper 最大的收获不是 "知道 CSA/HCA/mHC 长什么样", 而是看清一个 frontier lab 真正的工程武器库长什么样。这才是这篇论文被低估的部分。

说到底，V4 真正难复现的不是 CSA，而是 MegaMoE、TileLang、batch invariance 与 KV cache 系统的复合。这是 systems engineering 的胜利，不是 architecture engineering 的胜利。

※ 本文基于 DeepSeek-V4 技术报告（2026 年 4 月）的批判性阅读，参考了 LMSYS / SGLang day-0 集成博客、Thinking Machines Lab 的 [Defeating Nondeterminism in LLM Inference](#)、DeepGEMM 仓库 PR #304、TileLang 论文（Wang et al. 2026 / OpenReview Jb1WkNSfUB）、HuggingFace deepseek-ai/DeepSeek-V4-Pro 模型卡，以及 Microsoft / OpenAI 等关于 MXFP4 的公开技术博客。

※ 文中 B200 算力 / NVLink 5 带宽 / TDP 数字基于公开资料近似估算，实际部署中 V4-Pro 训练所用硬件配置以 DeepSeek 官方公开为准。